Combien de bits ?

Taille d'une écriture binaire, somme, produit et dépassement

Séquence
SQ01

Partie
P03

Niveau
Première NSI

Objectifs

- **Déterminer** le nombre de bits nécessaire pour écrire un entier, en l'encadrant entre deux puissances de 2.
- **Prévoir** la taille d'une somme et d'un produit.
- **Expliquer** un dépassement de capacité sur une taille fixe, et le simuler en Python.

Prérequis : poids des bits, divisions successives, tableau de suivi (partie 2).

Le problème

	Taille d'un entier
Machine des nombres	8 bits : 255 au maximum
Python	variable : `2 ** 100` s'affiche en entier
Processeur, fichier, réseau, capteur	**fixée à l'avance** : 8, 16, 32 ou 64 bits

Deux questions pour un informaticien :

1. Combien de bits un nombre demande-t-il ?
2. Que se passe-t-il quand il n'y en a pas assez ?

Chaque bit double les possibilités

Bits	Valeurs	Plus grand entier	Usage
1	2	1	vrai / faux
4	16	15	un chiffre hexadécimal
8	256	255	un octet, une couleur
16	65 536	65 535	numéro de port TCP
32	4 294 967 296	≈ 4,3 milliards	adresse IPv4
64	$\approx 1{,}8 \times 10^{19}$	$\approx 1{,}8 \times 10^{19}$	processeur actuel

Avec *n* bits : **2^n valeurs**, de 0 à **$2^n - 1$**. Repère : $2^{10} = 1024 \approx 1000$.

Combien de bits pour écrire N ?

N demande n bits quand $\mathbf{2^{n-1} \leq N < 2^n}$: on encadre N entre deux puissances de 2 consécutives.

N	Encadrement	Bits	Écriture binaire
13	$8 \leq 13 < 16$	4	`1101`
200	$128 \leq 200 < 256$	8	`11001000`
255	$128 \leq 255 < 256$	8	`11111111`
256	$256 \leq 256 < 512$	9	`100000000`

C'est la **longueur** de l'écriture binaire, sans zéros à gauche.

En Python

```
>>> bin(200)
'0b11001000'
>>> len(bin(200)) - 2
8
>>> (200).bit_length()
8
```

- `len(bin(n)) - 2` : longueur du texte, moins le préfixe `0b`.
- `n.bit_length()` : directement le nombre de bits.
- Les parenthèses autour de `200` évitent que Python lise `200.` comme un nombre à virgule.

ACTIVITÉ

Exercice 3.1 — Compter les divisions

Papier · seul puis à deux · 10 min

À produire : le tableau de suivi de `nb_bits(13)`, puis les valeurs prévues pour 15 et 16.

```
def nb_bits(n):
    compteur = 1
    while n > 1:
        n = n // 2
        compteur = compteur + 1
    return compteur
```

Question à trancher à deux : pourquoi `compteur` commence-t-il à 1 ?

Le résultat d'un calcul

Somme et produit : raisonner sur le pire cas

Opération	Plus grand résultat	Taille
Somme de deux entiers de n bits	$2^{n+1} - 2$	au plus **n + 1** bits
Produit n bits × m bits	$(2^n - 1)(2^m - 1) < 2^{n+m}$	au plus **n + m** bits

Sur 8 bits	Calcul	Bits nécessaires
Somme maximale	255 + 255 = 510	9
Produit maximal	255 × 255 = 65 025	16

Quand les bits manquent : le dépassement

Retenue	8 bits conservés
`1`	`00000000`

255 + 1 sur 8 bits : le 9e bit est **perdu**, le résultat vaut **0**.

- On parle de **dépassement de capacité** (*overflow*).
- La machine compte « modulo 256 », comme un compteur kilométrique qui repasse à zéro.
- En Python : `(a + b) % 256` simule une addition sur 8 bits.

ATTENTION

Des dépassements bien réels

Système	Taille	Conséquence
Pac-Man (1980)	niveau sur 8 bits	niveau 256 : moitié de l'écran illisible
Ariane 5, vol 501 (1996)	entier de 16 bits	calculateur arrêté, lanceur perdu
YouTube, *Gangnam Style* (2014)	entier signé de 32 bits	limite de vues proche : passage à 64 bits
Horloge Unix	entier signé de 32 bits	limite le 19 janvier 2038

Vérifier qu'un résultat tient dans sa taille : c'est de la **mise au point**.

ACTIVITÉ

Exercice 3.2 — À la main

Papier · seul · 10 min

À produire : le nombre de bits, justifié par un encadrement entre puissances de 2.

1. 100, 255, 256, 1000.
2. Un million, avec $2^{10} \approx 1000$.
3. Somme et produit de deux entiers de 8 bits ; produit d'un entier de 8 bits par un entier de 4 bits.

Exercice 3.3 — L'énoncé

Une machine à 8 bits ne garde que les huit bits de droite d'un résultat.

```
def addition_8_bits(a, b):
    """Somme de a et b sur 8 bits (a, b entre 0 et 255)."""
    ...

def deborde(a, b):
    """True si a + b ne tient pas sur 8 bits."""
    ...
```

Tests : `addition_8_bits(100, 50) == 150` · `addition_8_bits(255, 1) == 0` · `addition_8_bits(200, 100) == 44` · `deborde(255, 1) == True` · `deborde(200, 55) == False`

ACTIVITÉ

Exercices 3.3 et 3.4 — Simuler une machine à 8 bits

Sur ordinateur · seul · 15 min

Où : cours.siovision.fr › NSI 1G › « Du métier Jacquard au binaire » › exercice 3.3. Éditeur et tests dans la page ; EduPython en repli.

1. `addition_8_bits`, puis `deborde`, jusqu'à ce que tous les tests passent.
2. Le QCM 3.4, juste en dessous sur la même page.
3. Ensuite : l'atelier Pyxel « Le compteur 8 bits » (lien dans l'exercice 3.3 suite).

Indice : garder les huit bits de droite, c'est prendre le reste de la division par 256.

À RETENIR

À retenir

1. Avec n bits : 2^n valeurs, de 0 à $2^n - 1$.
2. N demande n bits quand $2^{n-1} \leq N < 2^n$; en Python, `n.bit_length()`.
3. Somme de deux entiers de n bits : au plus $n + 1$ bits ; produit $n \times m$ bits : au plus $n + m$ bits.
4. Sur une taille fixe, un résultat trop grand **déborde** : on garde les bits de droite.
5. Question ouverte pour la partie 4 : comment écrire un entier **négatif** ?